www.editada.org

International Journal of Combinatorial Optimization Problems and Informatics, 18(1), 2026, 17-25. ISSN: 2007-1558.
Doi

From Synthetic Seeds to Federal Records: An Extended Dimensional Data Warehouse for Municipal Public Works with a Measured Lakehouse Layer

Gabriel Hurtado Avilés[1], Roberto Eswart Zagal Flores[1], Uriel González Casiano[1], Marco Tulio Maldonado Mejía[1]

[1] Escuela Superior de Cómputo (ESCOM), Instituto Politécnico Nacional, Mexico City, Mexico

ghurtadoa@ipn.mx, rzagalf@ipn.mx
{ugonzalezc2400, mmaldonadom2200}@alumno.ipn.mx
gabrielhuav@gmail.com, gonzalezcasianouriel@gmail.com, marcotuliomm.s45@gmail.com

✉Corresponding author: gabrielhuav@gmail.com

Abstract. Municipal public-works records are scattered across spreadsheets and transactional systems. This article presents a dimensional Data Warehouse for monitoring municipal public works —ten dimensions, five of them Slowly Changing Dimension Type 2, two fact tables and a REST API— and, in this extended version, the lakehouse layer that the conference version left as future work. The layer keeps nine official files on federally funded projects for 2020–2025 in an S3-compatible object store, as Apache Iceberg tables queried by DuckDB, and loads 1,537,019 project-year records into the warehouse without altering its original tables. Seven paired queries return byte-identical results on PostgreSQL and on the lake; the lake is faster on national aggregations, the warehouse on single-state queries. Over synthetic ground truth, a per-type cost criterion raises the detection rule's precision from .46 to .51 and its recall from .35 to .41.

Keywords: Data Warehouse, Lakehouse, Apache Iceberg, Slowly Changing Dimensions, Open Government Data, Public Works

Article information
Received:
Accepted:

1 Introduction

Transparency in the management of municipal public works is one of the most persistent challenges of digital governance in Mexico. Public-works departments simultaneously manage contractors, budgets, field supervisors, progress photographs, monthly reports, and hand-over processes; yet most municipalities operate with distributed file systems, spreadsheets, or, at best, isolated transactional systems that preclude historical analysis, early delay detection, and informed citizen participation.

The *lakehouse* paradigm (Zaharia et al., 2021) offers a reference response to this fragmentation: it unifies the flexibility of Data Lake object storage (unstructured data, direct ingestion, low cost) with the analytical capability of a Data Warehouse (star schema, slowly changing dimensions, OLAP views). This work does not implement a full enterprise lakehouse: given the data volume at municipal scale, we build a *dimensional Data Warehouse* as the analytical core and use object storage only as a repository of unstructured evidence linked to it. Full lakehouse convergence —an open table format queried directly over the lake— was proposed as an evolution path and is implemented in this version (Section 8).

This article presents a prototype for the geospatial monitoring of public works in Temascaltepec, State of Mexico (35,014 inhabitants in the 2020 Census; Instituto Nacional de Estadística y Geografía [INEGI], n.d.-a).

This extended version keeps that design and implements what the conference version left as future work. The object store, which only held evidence files, now carries a lakehouse layer over official records: the federally funded projects that states and municipalities reported to the Ministry of Finance for 2020 to 2025 (Secretaría de Hacienda y Crédito Público [SHCP], n.d.), in all 32 states, joined with the 2020 Census (INEGI, n.d.-a), the 2023 census of municipal governments (INEGI, n.d.-b) and the 2020 social deprivation index (Consejo Nacional de Evaluación de la Política de Desarrollo Social [CONEVAL], n.d.). The version also recomputes the cost criterion per work type, as the failure analysis of the conference version suggested, projects the records to the Open Contracting for Infrastructure Data Standard and to an RDF graph, and corrects three figures of the conference version (Section 7.7).

© Editorial Académica Dragón Azteca S. de R.L. de C.V. (EDITADA.ORG). All rights reserved.

Research Questions

RQ1. Can a dimensional Data Warehouse deployed on commodity managed services (PostgreSQL/Supabase plus S3-compatible object storage) provide auditable historical traceability of municipal public works at the scale of a rural Mexican municipality, without the enterprise platforms normally associated with lakehouse architectures?

RQ2. Which dimensional design decisions —fact grain, choice of Slowly Changing Dimension type per entity, and view materialisation strategy— are required so that delay detection, budget alerting and citizen participation can all be served from a single analytical schema?

RQ3. How effective is a threshold rule expressed over the warehouse's analytical views at recovering anomalies in public-works execution, compared with an unsupervised baseline, and which anomaly classes does it fail to recover?

RQ4. Can the warehouse evolve into a lakehouse over official public-works records without altering its schema, and which questions does each architecture answer better?

The four structure both the design and the evaluation: RQ1 is addressed by the architecture and warehouse design (Sections 3 and 4) and by the cost and deployment discussion (Section 10); RQ2 by the grain, SCD and view-strategy analysis (Section 4); RQ3 by the evaluation of the detection module (Sections 7.4 and 7.5); and RQ4 by the lakehouse layer and its evaluation (Sections 8 and 9).

Contributions

The contribution is threefold: (1) a reference architecture centered on a dimensional *Data Warehouse* with slowly changing dimensions, complemented with S3-compatible object storage for evidence and exposed through a REST API (RQ1); (2) a geospatial map acting as a graph-like exploration interface over works, communities, budgets and proposals (RQ2); and (3) a non-circular evaluation of the detection module against latent causes, with a failure analysis by class (RQ3). This extended version adds (4) a lakehouse layer over the 2020–2025 federal records, loaded additively into the warehouse and checked against it query by query (RQ4), and (5) a per-type cost criterion that addresses the failure diagnosed in Section 7.4.

Over a seeded test scenario, the prototype organizes 1,247 simulated public works with an accumulated budget of $127.4 million pesos (Section 7 details the full population). Except in Sections 8 and 9, which measure official records, every figure in this article comes from synthetic data: the warehouse population from a seeded generator, the detection results from ten seeds and four prevalences. They characterise the behaviour of the system, not the municipality of Temascaltepec, and no claim about real public-works execution there is made or implied. Section 9.3 reports what the official records list for Temascaltepec, which is not a claim about how its works were executed either.

The remainder of the article is organized as follows: Section 2 reviews related work and contrasts the proposal with comparable systems; Section 3 describes the architecture; Section 4 details the warehouse; Section 5 describes the object storage layer; Section 6 presents the geospatial map; Section 7 reports results; Section 8 builds the lakehouse layer over official records and Section 9 evaluates it; and Section 10 concludes.

2 Related Work

2.1 Data Lakehouse and Architecture Convergence

Zaharia et al. (2021) coined the term *lakehouse* to describe an architecture that combines the open, flexible storage of a Data Lake with the ACID transaction guarantees, schema control, and analytical performance of a Data Warehouse. The transactional layer that makes this convergence possible was described by Armbrust et al. (2020), who show how an open table format over cloud object stores can provide ACID semantics and time travel without a dedicated database engine; this is precisely the mechanism our object layer does not yet implement. Databricks (Barreto et al., 2021) extended this model to the geospatial domain, integrating vector and raster data on a unified platform.

In the present work we adopt the spirit of the lakehouse without requiring Apache Spark or Delta Lake: S3-compatible object storage (Cloudflare R2) acts as the evidence layer and PostgreSQL/Supabase as the warehouse layer, connected by a REST API and by database triggers rather than by an external ETL process. We stress that, in its current state, the object store is a binary-evidence repository and not a queryable analytical lake: no open table format (Iceberg/Delta/Hudi) and no engine querying over the lake are used. We therefore position the system as a dimensional Data Warehouse with object storage, and reserve full lakehouse convergence for future work.

2.2 Geospatial Knowledge Graphs

Knowledge graphs (KGs) have proven useful for integrating heterogeneous government data. Hogan et al. (2021) provide a comprehensive survey of the state of the art in KGs, highlighting their application to georeferenced entities. Janowicz et al. (2022) present KnowWhereGraph, one of the largest public geospatial graphs, which links human and environmental data under FAIR principles. In the open-government-data domain, Espinoza-Arias et al. (2020) document the case of Zaragoza,

where a municipal knowledge graph connects public services, infrastructure, and demographic data. Rajabi et al. (2023) systematize the construction of knowledge graphs over open government data using Semantic Web technologies.

Our geospatial map adopts an analogous *relational* approach at a rural municipal scale: public works, communities, budgets, and citizen proposals constitute the entities, while reports and photographs represent linked evidence. The distinction with respect to the works above is deliberate and is stated explicitly in Section 6: the graph is implicit in the foreign keys of the dimensional model and is materialised only at the interface level. Publishing the schema as an OWL vocabulary, which would place the system in the same family as Espinoza-Arias et al. (2020), Janowicz et al. (2022) and Rajabi et al. (2023), is the first item of our future work.

2.3 Open Government Data and Infrastructure Disclosure Standards

The literature on Open Government Data (OGD; Lourenço, 2015) emphasizes that publishing public-infrastructure data in reusable formats increases accountability and facilitates the detection of irregularities. However, most OGD systems in Latin American municipalities are limited to static download portals, without analytical capability or integration with field data.

A directly relevant body of practice is the family of infrastructure disclosure standards. The Open Contracting for Infrastructure Data Standard (OC4IDS; Open Contracting Partnership & CoST – The Infrastructure Transparency Initiative, n.d.), developed by the Open Contracting Partnership with CoST — the Infrastructure Transparency Initiative, defines a JSON schema for publishing project- and contract-level information across the infrastructure lifecycle. OC4IDS and our proposal are complementary: OC4IDS specifies *what* a government should disclose and in which interchange format, whereas we address *how* a small municipality can produce, historise and analyse that information internally. A natural convergence (Section 8.3) is to expose the warehouse as an OC4IDS-conformant endpoint.

Four bodies of work bear directly on the design. Spatial OLAP (Rivest et al., 2005) has combined multidimensional analysis with cartographic navigation since the mid-2000s; our map is a SOLAP client in all but name, differing in scope rather than kind, since geometry is a display attribute of `dim_region` and not a hierarchy to drill along. The trade-offs of slowly changing dimensions are surveyed by Faisal and Sarwar (2014); ours is not the mechanism but its application at audit granularity. Ambituuni (2026) shows that disclosure alone does not curb irregularity, which is the argument for coupling publication with analysis. And indicator-based detection over procurement records has an empirical tradition (Fazekas & Kocsis, 2020) —the family our C1–C3 rule belongs to, and against which its modest performance (Section 7.4) should be read. This same approach of consolidating official open data into a queryable analytical artifact has been applied before in other government domains in Mexico, such as seismic risk (Villa Vargas et al., 2026), a methodological antecedent transferred here to the monitoring of public works.

2.4 Positioning with Respect to Comparable Systems

Set against representative systems —the geospatial knowledge graphs, the OC4IDS disclosure standard, the geospatial-lakehouse architecture and the download portal typical of Latin American open-government sites— the proposal differs on two axes and coincides on the rest. Those systems variously offer a formal semantic layer, a richer interchange format or far greater scale; none preserves entity state at audit granularity, and none manages binary field evidence as a first-class citizen linked back to the analytical store.

The semantic-web systems (Espinoza-Arias et al., 2020; Janowicz et al., 2022; Rajabi et al., 2023) provide a formal layer we lack, but none answers “what did this record look like the day the budget was modified?”, which is what SCD 2 provides; and in OGD portals and OC4IDS implementations documents are referenced by URL without a managed lifecycle. The intersection of *temporal auditability* and *evidence linkage* at municipal scale is where this work sits. That is the position of the proposal, not a claim of vacancy: establishing that no comparable system exists would require a systematic review we have not carried out. This work fills the intersection of *temporal auditability* and *evidence linkage* at a scale where enterprise platforms are not economically viable.

3 System Architecture

Fig. 1 illustrates the four layers of the proposed architecture, which follows a layered pattern with a clear separation of responsibilities. It is described as a reference architecture and prototype: the analytical core is the dimensional Data Warehouse, and the object store operates as an evidence layer. The subsections below correspond one-to-one to the four layers labelled in the figure, so that the terminology of figure and text coincides.

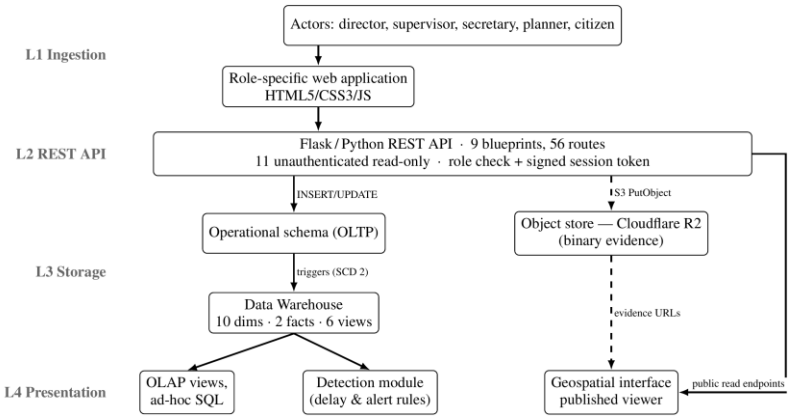


Fig. 1. System architecture (warehouse-centric), redrawn in English. The four layers L1–L4 correspond to Sections 3.1–3.4. Solid arrows: structured-data flow; dashed arrows: unstructured (binary) evidence flow. Released role-specific pages read operational endpoints; the warehouse serves detection and OLAP through `/api/analitica`. The geospatial viewer is part of the released artefact.

3.1 Layer 1: Ingestion

The actors (directors, supervisors, secretaries, planners and registered citizens) interact with a web application built in HTML5/CSS3/JavaScript. Forms send structured data to the API (text, coordinates, amounts) and binary files (JPG/PNG, PDF) to the same endpoint, which routes each to its destination: the relational insert is transactional, while the pairing with object storage is compensated rather than atomic (Section 5).

3.2 Layer 2: REST API

A Flask/Python REST API organised in nine blueprints declares 56 routes, plus a health check on the application. Eleven declarations carry no authentication decorator and expose only `GET`: five under `/api/public/`, two under `/api/analitica/` serving the warehouse views of Section 4.3, and four citizen-participation reads, one of which returns data only against a valid session token. That analytical surface is public by design. Counts are of route declarations, so a route accepting two methods counts once. Access control for staff is a `@require_auth` decorator validating a role identifier carried in request headers; the citizen module uses HMAC-signed session tokens, and passwords are PBKDF2-HMAC-SHA256 hashes with a 16-byte salt. The scheme is deliberately lightweight and is *not* a full token-based authentication service: hardening it —token expiry, rotation, and moving the staff role check out of a header— is listed among the limitations in Section 10.2.

3.3 Layer 3: Storage

Object store (Cloudflare R2). An S3-compatible object store that receives unstructured evidence through direct ingestion. Paths follow a *hive-partitioned* convention, shown in Equation (1):

```
works/{work_id}/reports/{year}-{month}/{ts}_{slug}.{ext}        (1)
```

This structure enables direct exploration by work and period; each object's metadata (public URL, original name, MIME type, size in bytes, upload date) is also persisted in a warehouse-side table, creating a bidirectional link between the two layers.

Data Warehouse (PostgreSQL/Supabase). An analytical layer with a star schema, two fact tables, and ten dimensions (Section 4). The warehouse is populated from the operational schema by database triggers rather than by an external ETL job: eight triggers synchronise the dimensions (applying SCD 2 versioning where required) and four triggers emit rows into the audit fact table when a progress report, a cost record, an image upload or a citizen vote is registered. The warehouse uses range partitioning by year on the audit fact table to optimize historical queries.

3.4 Layer 4: Presentation

The released presentation layer consists of role-specific HTML/CSS/JavaScript pages that consume the REST API. Analytical consumption —the six views of Section 4.3, ad-hoc OLAP queries and the detection module of Section 3.5— is served directly from the warehouse through `/api/analitica`. The geospatial viewer of Fig. 3 is published alongside the rest of the artefact and reads a frozen projection of the same synthetic dataset, so map and workspaces never disagree.

3.5 Analytical Detection Module

The module is stateless, reads the warehouse and emits three artefacts: a *delay set* (`v_delayed_works`) of every work whose latest monthly row carries the delay flag with its slippage; an *alert stream* (`v_audit_alerts`) classifying audit

events into five categories through a `CASE` expression over the fact table; and an *anomaly set* (`v_anomalies_detection`) applying the C1–C3 criteria below to each work–month row. The third, not the other two, is what Section 7.4 evaluates.

The rule applied to a work–period record flags it as anomalous if *any* of three operational criteria holds:

C1. *Cost outlier*: the executed budget deviates from the mean by more than three standard deviations, $|b_i - \mu_b|/\sigma_b > 3$.

C2. *Extreme delay*: the recorded slippage exceeds 120 days.

C3. *Progress inconsistency*: physical progress below 30% together with financial progress above 80%.

The corresponding anomaly score is $s_i = \max\left(\frac{|b_i-\mu_b|}{\sigma_b}, \frac{d_i}{120}\mathbb{1}[d_i > 120], 3.5 \cdot \mathbb{1}[\text{C3}]\right)$, where d_i is the slippage in days; the module flags $s_i > 3$. The score is used for the ranking-based metrics reported in Section 7.4. C1–C3 are implemented as `v_anomalies_detection` over the monthly fact table and served read-only at `/api/analitica/anomalias`, so the rule evaluated in Section 7.4 is the rule an operator runs.

3.6 A Note on Identifiers

This article uses English identifiers for tables, columns and views; the published implementation uses Spanish ones, as it was built with and for a Spanish-speaking municipal office. The correspondence is mechanical —`fact_audit_events` is `fact_eventos_auditoria`, `v_delayed_works` is `v_obras_retraso`, `dim_work` is `dim_obra`, and so on— and the repository README tabulates every pair, so that a reader inspecting the code can locate each object referenced here. This mapping is a prerequisite for the reproducibility claims of Section 7.

4 Data Warehouse Design

The warehouse adopts a dimensional star model (Kimball & Ross, 2013) with two fact tables and ten dimensions, shown in Fig. 2 with the SCD type of each.

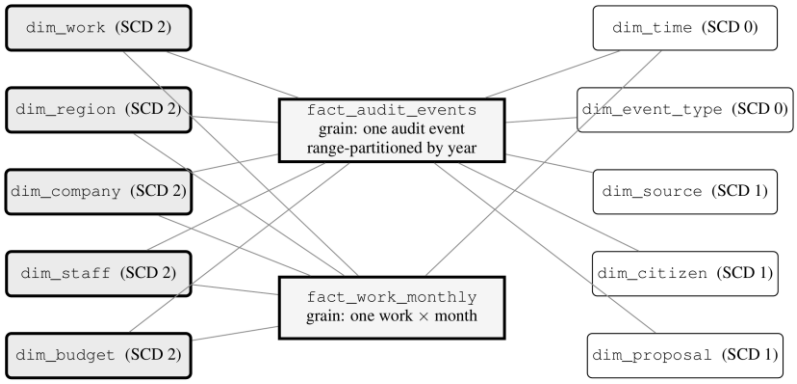


Fig. 2. Dimensional star schema, redrawn for legibility. Two fact tables (`fact_audit_events`, range-partitioned by year, and `fact_work_monthly`) are connected to the ten dimensions through surrogate keys. The five Slowly Changing Dimensions of Type 2 (shaded, left) are those that carry the historical traceability of the works. The monthly fact table is conformed only with the five SCD 2 dimensions and the time dimension.

4.1 Fact Tables and Grain

We state the grain of each fact table explicitly, since it determines every subsequent aggregation and is a prerequisite for reproducing the design.

`fact_audit_events` has a grain of *one row per audit event*: a single, atomic, immutable action performed on a work by an identified actor at an identified instant. It records events classified into 17 types (work creation, work modification, state change, progress report, cost record, budget record, budget modification, image upload, permit issue, hand-over record, contractor selection, funding assignment, supervisor assignment, role change, proposal registration, vote cast and session start) and is partitioned by year. The seeded scenario writes 8,934 events directly; the synchronisation triggers emit more as reports, costs, images and votes are registered, so a freshly seeded instance holds 55,740 (Section 7.7). Each row carries foreign keys to the ten dimensions, financial metrics, progress metrics, and audit flags.

`fact_work_monthly` has a grain of *one row per work per calendar month*. It is a periodic snapshot table: for each work and month it stores the total budget, the accumulated cost, the remaining balance, the percentage executed, the physical progress, the number of reports and images registered, and a boolean delay flag with the associated slippage in days. Because

the grain is a snapshot rather than a transaction, aggregating cost across months requires taking the last snapshot of the period rather than a sum; the analytical views encapsulate this rule so that consumers cannot get it wrong.

4.2 Slowly Changing Dimensions

The SCD 2 dimensions (Kimball & Ross, 2013) (five in total: works, regions, companies, staff, and budget) maintain history through validity columns (`effective_date`, `expiration_date`, `is_current`), which makes it possible to reconstruct the exact state of any work at an arbitrary point in time.

Three representative use cases motivate this choice, none answerable with a Type 1 dimension. In a *budget-modification audit*, `dim_budget` closes the current version and opens a new one, so an auditor can ask which amount was in force on the date a payment was authorised, not merely the amount in force today. Under *contractor reassignment*, `dim_company` preserves both versions, so progress recorded before the change stays attributed to the original contractor —a precondition for any performance analysis by contractor. Under *territorial reorganisation*, where communities are merged or renamed, `dim_region` versioning prevents a historical investment series from being silently rewritten.

The remaining five do not version. `dim_time` and `dim_event_type` are fixed catalogues (Type 0). `dim_source` is Type 1: it carries running totals of works financed and amount assigned, which are overwritten as they change. `dim_citizen` and `dim_proposal` are Type 1 because only their current state is analytically meaningful and, since they hold names and national identity codes, retaining superseded versions would create a personal-data liability with no analytical return. The distribution is five Type 2, three Type 1 and two Type 0. `scripts/verificar_scd.py` checks the schema against this classification, so the two cannot drift apart.

4.3 Analytical Views and Refresh Strategy

Six analytical views derived from the fact tables support analytical consumption: (1) **`v_delayed_works`** (works with a schedule slip and days of delay); (2) **`v_audit_alerts`** (alerts classified as HIGH_AMOUNT, CANCELLED_WORK, BUDGET_CHANGE, LATE_EVENT and REVIEW); (3) **`v_work_traceability`** (full SCD 2 version history of a work); (4) **`v_citizen_participation`** (votes and proposals by region); (5) **`v_budget_executed`** (accumulated budget execution by funding source); and (6) **`v_anomalies_detection`** (the C1–C3 criteria of Section 3.5 evaluated per work and month, with one column per criterion).

These are standard SQL views rather than materialized ones. The choice is deliberate and determines the refresh strategy. At the volumes involved —of the order of 10^4 fact rows for 35,014 inhabitants— the views execute in tens of milliseconds over the partitioned fact table, so materialisation would buy little while introducing a staleness window; and since alerts must be actionable the moment a supervisor uploads a report, a live view is semantically preferable, with no refresh job to schedule or to fail. Should the deployment scale to several municipalities, the migration path is to materialise the aggregation-heavy `v_budget_executed` and `v_citizen_participation` with `REFRESH MATERIALIZED VIEW CONCURRENTLY` after each nightly load, keeping the two alert-bearing views live.

5 Object Storage Layer: Evidence Storage

The object layer uses Cloudflare R2, a service compatible with the Amazon S3 API, selected for its zero egress cost and reduced global latency. In its current state it functions as an object repository for binary evidence (images and PDFs) linked to the warehouse, and not as a queryable analytical lake: analytical queries are resolved in the Data Warehouse, not over R2. Adopting an open table format (Armbrust et al., 2020) that enables direct queries over the lake was proposed as future work and is implemented in Section 8. The path design follows the *hive-partitioned* convention (Barreto et al., 2021) of Equation (1).

Ingestion is direct from the web form: the supervisor attaches images or PDFs when creating a progress report; the API computes the object key, uploads it through the S3 SDK, records the metadata in the warehouse, and returns the public URL. This is not an atomic transaction across the two stores: object storage and PostgreSQL cannot enlist in one. The implementation compensates —on a database failure it issues a best-effort delete of the objects just uploaded— so a crash between the two steps can still leave an orphaned object. Reconciling them would need a sweep against the metadata table, which is not implemented.

6 Geospatial Interface Prototype

The geospatial display is an *implicit* graph at the interface level —the relationships are derived from the foreign keys of the dimensional model— and not a formal knowledge graph with an ontology or an RDF triple store; publishing the schema as a linked vocabulary is addressed in future work.

Fig. 3 shows the main interface, with markers color-coded by the state of the work and the technical-card popup that displays physical and financial progress indicators when a node is selected. It is part of the released artefact, reachable from the navigation bar of every page. Each marker sits at the real coordinates of its community, resolved from OpenStreetMap, with a deterministic offset of at most 150 m so that two works in the same community do not overlap. The community is therefore real; the exact site within it is not surveyed.

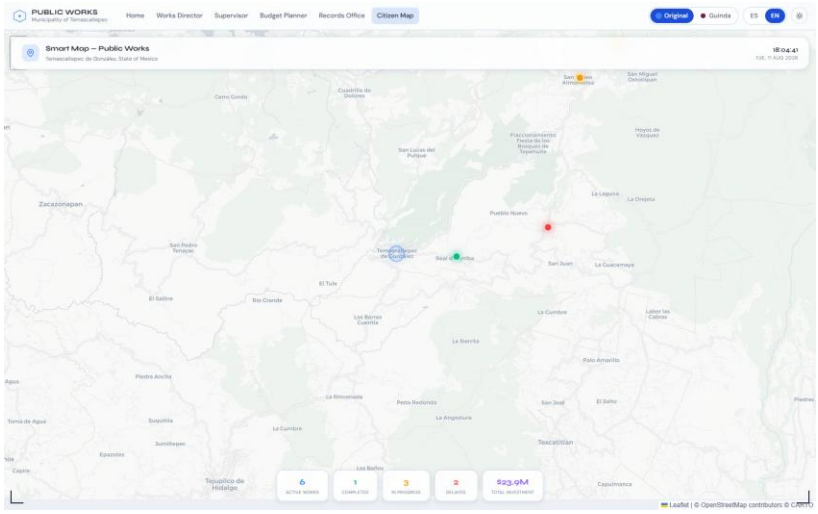


Fig. 3. The published geospatial viewer, in light mode. Markers are color-coded by state —green completed, amber in progress, red delayed— over the six works of the demonstration dataset, with the indicator panel below. Live at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/mapa/?modo=claro&idioma=en; captured in a 1500×938 viewport. The viewer opens on a fixed centre and zoom over the municipality, so the window size is what decides how much surrounding territory is in frame. Screen capture of the authors' own artefact; base map © OpenStreetMap contributors (ODbL) and © CARTO, rendered with Leaflet.

6.1 Management and Visualization Components

- **Marker layer:** each active work is represented by a marker color-coded by state —green (completed), amber (in progress), red (delayed)— derived from the work's latest progress report.
- **Technical card (popup):** selecting a work displays its file number, name, community, total budget, assigned company, date range, a physical-progress bar, and an expandable section for images from the object layer.
- **Analysis layer:** highlights delayed works and indicates the days of slippage (criterion C2 of Section 3.5); the dual physical/financial percentage detects works with high budget execution and low physical progress (criterion C3), a potential indicator of irregularities.

7 Results and Evaluation

7.1 Scope of the Evaluation

Measured results and projections are kept strictly apart in what follows. We label every figure in this section as **measured** (obtained by executing published code or auditing the published artefact, and reproducible by a third party: Sections 7.2 to 7.5 and 7.7) or **projected** (an expected operational outcome not yet observed in any deployment: Section 7.6). Outside Sections 8 and 9, no number in this article is a measurement taken on real municipal data.

Three synthetic populations appear here and must not be conflated. The *published site* holds six works in the visitor's browser tab and supports only interface claims and the audit of Section 7.3. The *seeded warehouse* —1,247 works, 55 communities, $127.4 M, 29,928 monthly fact rows— supports the structural claims and the latencies of Section 7.2. The *detection dataset*, a separate simulation of about 2,580 work–period records per seed, supports Table 2 and nothing else. Sections 8 and 9 add a fourth population, of official records, which is never combined with the three synthetic ones.

7.2 End-to-End Latency (Measured)

Latency is measured against the reference stack in `docker/` —PostgreSQL and the Flask API in containers, loaded with the seeded population— and is reproducible with `scripts/medir_rendimiento.py`. Table 1 reports the measurements.

Table 1. Latency measured against the reference stack, with `scripts/medir_rendimiento.py --repeticiones 100`. API rows are 100 serial requests over the seeded population of 1,247 works; view rows are the server execution time reported by `EXPLAIN ANALYZE` over 25 runs; throughput is 100 requests across ten concurrent clients.

Operation	Mean	p95
`/api/public/obras` (766 kB response)	566 ms	610 ms
`/api/public/resumen`	544 ms	589 ms
`/api/public/regiones`	10.9 ms	17.7 ms
`/api/analitica/anomalias`	18.3 ms	33.4 ms
`v_delayed_works`	2.0 ms	2.2 ms
`v_anomalies_detection`	23.8 ms	24.5 ms
`v_budget_executed`	5.2 ms	5.4 ms
Insert firing the SCD 2 trigger	0.1 ms	0.1 ms
Sustained throughput, 10 clients		3.4 req/s

Environment: Windows 10 host, Docker 29.6, PostgreSQL 16.14 in a container with 12 CPUs, Flask behind gunicorn with two synchronous workers, 1,247 works, 14,885 progress reports and 29,928 monthly fact rows.

The result does not flatter the design uniformly. The warehouse answers in single-digit milliseconds: the delayed-works view runs in 2 ms and an insert firing the SCD 2 synchroniser costs 0.1 ms, so neither the star schema nor the versioning triggers are the limit at this scale. The delivery layer is where the time goes. `/api/public/obras` takes 566 ms because it serialises the entire portfolio in one unpaginated response, and sustained throughput is 3.4 req/s, which is what two synchronous workers and a 566 ms response give. Pagination and more workers are the remedy, and neither touches the dimensional design. Map load, gallery rendering and object-store latency are not reported: measuring them needs a deployed map backend and an R2 bucket the artefact does not include, and a gap is preferable to an uncheckable number.

The seeded warehouse holds 1,247 works across 55 communities, 8,934 audit events, 3,421 photographic pieces of evidence, $127.4 million pesos in budget, 2,156 citizen proposals and 8,723 votes cast.

7.3 Delivery of the Published Frontend (Measured)

The static artefact is permanently online, so its delivery can be audited by anyone against a public URL. A Lighthouse audit of its two entry points, taken as the median of three runs each on 11 August 2026 (Lighthouse 12.8.2, headless Chrome 151, emulated mobile with simulated throttling, tag `v1.7.3-icokg2026`), gives the landing page 85 for performance and the map 72, with 100 for accessibility, 96 and 93 for best practices and 90 and 100 for SEO; both pages leave the main thread idle and shift no layout, and first contentful paint is 3.2 s and 3.6 s. What separates them is delivery weight: render-blocking fonts and a CDN animation library on one side, a 533 kB bundle on the other. Reaching 100 on accessibility meant fixing what the audit reported —a brand link with no accessible name below 720 px, an attribution painted below 4.5:1, two icon-only buttons, and map markers that are `role="button"` elements with no text— rather than restating the score.

This audit measures the delivery of a static page. It executes no Flask, no PostgreSQL and no warehouse query, so it is evidence about the front end alone; the architecture is exercised in Section 7.2.

7.4 Evaluation of the Delay/Alert Detection Module (Measured)

Generator. Anomalies are now latent states of a simulated execution process, and what the detector sees are their observable consequences. A work is drawn with a contract price, a planned duration and a productivity parameter, and is then executed month by month under seasonal and remoteness effects; payment follows progress with a mobilisation advance, a lag and noise. Five latent causes are injected: a price inflated at award, an unproductive contractor, payments running ahead of execution, a work that stops, and a work contracted and paid but never built. Their magnitudes are drawn from distributions that *overlap the normal range*, so some anomalies are undetectable in the recorded data and some healthy works look irregular. That overlap is the condition for the evaluation to be informative.

Protocol. Ten seeds and four latent prevalences (5, 10, 15 and 20%); the main configuration is 15%, which yields about 2,580 work–period records per seed of which 17.2% are anomalous. We report the mean over seeds with a 95% confidence interval. The baseline is an Isolation Forest (Liu et al., 2008) (100 estimators, contamination set to the latent prevalence) over five normalised features. Because comparing a rule at a fixed threshold against a baseline at its default threshold compares two operating points rather than two detectors, the baseline is also evaluated at a *matched alert budget*: the *k* highest-scoring records, with *k* the number of alerts the rule raises.

Table 2. Detection performance at 15% latent prevalence. Mean over ten seeds with a 95% confidence interval; anomalies are latent process states, not detector predicates. Reproducible with the published scripts. The last two rows are added in this version (Section 7.5).

Method	Precision	Recall	F1	ROC-AUC	PR-AUC
Threshold rule (this work)	.462 ± .056	.351 ± .031	.397 ± .038	.630 ± .022	.310 ± .052
Isolation Forest (default)	.369 ± .053	.321 ± .031	.342 ± .037	.650 ± .029	.342 ± .052
Isolation Forest (matched budget)	.387 ± .054	.294 ± .033	.333 ± .038	.650 ± .029	.342 ± .052
Per-type rule R2, z > 3	.511 ± .056	.411 ± .033	.454 ± .040	.677 ± .023	.457 ± .047
Isolation Forest (R2 budget)	.377 ± .051	.303 ± .034	.336 ± .039	.650 ± .029	.342 ± .052

Results. The rule alerts on about a third of the anomalies and is right slightly less than half the time. It does not dominate the baseline: Isolation Forest ranks better on both AUCs at every prevalence tested, so as a scoring function the rule is the poorer of the two. Its advantage is the operating point —at the same alert budget it is more precise (.462 against .387) and recovers more (.351 against .294)— which matters because an oversight office reviews a fixed number of files per period, not a ranking. Both improve with prevalence (the rule from .217 to .557 precision between 5 and 20%).

Where it fails. Per-cause recall is uneven in a way that is diagnostic. The rule recovers unproductive contractors at .548 ± .059, ghost works at .474 ± .024 and stopped works at .386 ± .012, but overpricing at only .093 ± .051 and payments running ahead of execution at .061 ± .034. The reason is structural, not a matter of tuning. C1 compares an absolute amount against the mean *of its period*, so it flags large works rather than overpriced ones: a school inflated by 30% is still cheaper than an honest road. And no criterion examines the *gap* between physical and financial progress unless the physical figure falls below 30%, so a work paid steadily ahead of its execution passes unremarked if it eventually finishes. The rule is a schedule-failure filter with a narrow ghost-work check attached, not a cost-anomaly detector.

7.5 A Per-Type Cost Criterion (Measured)

Design. The failure analysis above suggests two repairs, and both were fixed before the evaluation was run. C1 is recomputed per work type: the logarithm of each work's budget is compared with the median of the works of its type and scaled by their median absolute deviation (×1.4826), so a work is flagged when it is expensive for its type rather than large in absolute terms; only upward deviations count. A fourth criterion, C4, flags a work whose financial progress exceeds its physical progress by more than 30 points, the gap that C3 only notices when physical progress is below 30%. R1 combines the per-type C1 with C2 and C3, and R2 adds C4; each runs at thresholds of 2, 2.5 and 3 over the ten seeds and four prevalences of Table 2, and the repository reports all six variants.

Results. At 15% prevalence, R2 at z > 3 has the highest precision and the largest areas under both curves of the six variants: precision .511 and recall .411, against .462 and .351 for the conference rule (Table 2). It now out-ranks Isolation Forest on both areas under the curve (.677 and .457, against .650 and .342), and at its alert budget Isolation Forest reaches .377 precision and .303 recall. Per cause, recall on overpricing rises from .093 to .192 (.291 at z > 2), on payments ahead of execution from .061 to .167, and on ghost works from .474 to .650. Overpricing remains the weakest class: an inflation that stays within the spread of its type still passes, so the per-type rule narrows the failure of Section 7.4 without closing it.

7.6 Illustrative Analytical Capability (Projected)

The warehouse answers questions a spreadsheet cannot: investment per inhabitant by community against the development index, budget concentration across funding sources, and participation rates by region. Over the seeded population these queries return coherent, plausible figures, which shows the analytical pipeline runs end to end. They are properties of the generator and nothing else: no claim about territorial inequality, budget concentration or savings in Temascaltepec is made or implied, and none can be until the model is populated with municipal records. Section 9.3 reports the first observations of this kind, from federal records.

7.7 Corrections to the Conference Version (Measured)

Re-running the published code exposed three figures of the conference version that do not hold; the text above now gives the measured values. Temascaltepec has 35,014 inhabitants in the 2020 Census (INEGI, n.d.-a), not 64,844, a figure that no script in the repository produces. A freshly seeded instance holds 55,740 audit events, not about 28,000: its 8,723 votes leave 17,993 vote events, because both the generator and the vote trigger record each vote. The response of `/api/public/obras` weighs 765,748 bytes, not 753 kB. `scripts/cifras_capitulo.py` measures the three against the reference stack.

8 A Lakehouse Layer over Official Records

8.1 Sources and Layers

The layer starts from nine files, each pinned by size and SHA-256 in a manifest: the six annual files of federally funded spending that SHCP publishes for 2020 to 2025, with projects, contracts, georeferences, physical targets and progress, financing and observations (SHCP, n.d.); the locality results of the 2020 Census (INEGI, n.d.-a); the public-procurement module of the 2023 census of municipal governments, CNGMD (INEGI, n.d.-b); and the 2020 municipal social deprivation index (CONEVAL, n.d.). Together they weigh 2,136.3 MB. A download that no longer matches its hash stops the build, so the layer cannot drift silently when a publisher replaces a file.

The layers follow the bronze, silver and gold progression usual in lakehouse deployments. Bronze keeps the files byte for byte in an S3-compatible object store; SeaweedFS (Lu, 2025) stands in for Cloudflare R2, which exposes the same interface. Silver holds ten Apache Iceberg tables with a single schema across years, written with PyIceberg (Apache Software Foundation, 2025) and partitioned by fiscal year and state: 11.0 million rows, built in about 200 s. Each fiscal year is committed as its own snapshot, which records the hash of its source file, so any table can be read as it stood after a given year was loaded. Gold holds analytical tables in Parquet, each the output of one SQL file run by DuckDB (DuckDB Foundation, 2025), which reads the Iceberg tables directly from the object store, without Spark.

Silver keeps what each publisher reports, and corrections are made, visibly, in gold. Two Sonora records carry as amount paid the digits of their own folio, 390.8 billion pesos together: gold keeps them in a column of reported values and sets them to null in the column used for sums. Until 2023 a folio appears in two reporting periods of the same year, and gold keeps the later one. Approved amounts are never added across years, because a multi-year folio repeats its total every year. The user field of the observations, which identifies officials, does not reach silver, and the 564 bytes that the source encoding leaves undefined are replaced and counted rather than dropped.

8.2 Additive Integration with the Warehouse

The lake does not replace the warehouse; it feeds it. Three tables join the schema of Section 4 without changing any of the original ones: `dim_municipality` (`dim_municipio`), a Type 1 dimension of the 2,469 municipalities with population and deprivation level; `dim_federal_project` (`dim_proyecto_federal`), a Type 2 dimension with 1,157,395 versions of 967,495 folios; and `fact_project_year` (`fact_proyecto_ciclo`), a snapshot fact table of 1,537,019 project-year rows that reuses the time dimension of the conference version. A new version opens when a folio changes status, approved amount or classification from one year to the next, so the SCD 2 design of Section 4.2 now records an observed history rather than a seeded one: of the 356,914 folios present in more than one year, 137,644 (38.6%) change status and 60,696 (17.0%) change approved amount, and none changes classification. The row count and MD5 checksum of each of the 34 original tables are identical before and after the load.

8.3 Interoperability: OC4IDS and an RDF Graph

Two projections, scoped to the State of Mexico, implement item (2) of the conference future work. The first maps each investment project to OC4IDS 0.9.5 (Open Contracting Partnership & CoST – The Infrastructure Transparency Initiative, n.d.) —status, sector, period, budget, public authority, location, completion value, physical progress and contracting processes, naming suppliers only when they are companies— and validates the result against the official schema: 78,826 projects, with no errors. The exercise also measures the source: SHCP fills 15 of the 37 top-level fields of the standard, so purpose, milestones, documents and transactions cannot be disclosed from these records however well they are mapped.

The second materialises the warehouse tables as an RDF graph through an R2RML mapping run by morph-kgc (Arenas-Guerrero, 2026) and stored in Oxigraph (Pellissier Tanon, 2023): 2,821,739 triples in 68.8 s, 29.9 MB compressed. Each project version is linked to the previous one with `prov:wasRevisionOf`, so the history of Section 8.2 can be walked in SPARQL, and every georeferenced observation carries a GeoSPARQL geometry. Each of the five SPARQL queries in the repository has a SQL twin, and the two return the same SHA-256; the comparison caught two mapping errors —sums over missing values and integer version numbers written as decimals— before any result was reported. The namespace, under w3id.org, has not yet been registered, so its IRIs identify resources but do not resolve.

9 Evaluation over Official Records (Measured)

All measurements in this section were taken on one Windows workstation (version 10.0.26300), with PostgreSQL 16.14 in its default configuration and DuckDB 1.3.2 with 12 threads; the repository holds the script behind each figure and its output.

9.1 Equivalence and Latency

Seven questions were written twice, in SQL for the warehouse and in SQL for DuckDB over the lake, from national totals by year to the history of a single folio. In every pair the two results have the same SHA-256, computed over their rows in order (Table 3). Latency splits by access pattern. The lake is faster on five of the seven queries, including every national aggregation —21.2 against 552.9 ms for the projects flagged by each criterion, 12.1 against 285.3 ms for the amount paid per inhabitant by deprivation level— as expected of a columnar engine that scans compressed data in parallel. The warehouse is faster on the two queries restricted to one state or one municipality, which its indexes answer in 12.0 and 2.4 ms, against 17.4 and 33.0 ms on the lake. The two stores therefore complement each other: the warehouse keeps serving the interface, and the lake takes the analytical scans.

Table 3. Seven questions asked of the warehouse and of the lake: rows returned and latency in milliseconds, mean and p95 of 20 runs after one warm-up run. In every pair, both results have the same SHA-256.

Question	Rows	PostgreSQL mean	PostgreSQL p95	Lake mean	Lake p95
Projects and amounts by year	6	123.8	144.9	36.1	54.0
Spending by state and type, 2024	435	240.9	381.0	40.4	50.7
State of Mexico, 2024	125	12.0	13.4	17.4	40.0
Paid per head by deprivation, 2023	5	285.3	326.0	12.1	29.7
Flags by criterion and year	6	552.9	594.3	21.2	36.0
Status of one folio in each year	5	61.8	66.7	35.1	53.9
Temascaltepec by year	6	2.4	4.4	33.0	52.1

9.2 Lakehouse Properties

The properties that justify a lakehouse were checked rather than assumed. *Partition pruning:* a filter on one year and one state plans 1 of the 192 data files of the projects table, 1.3 of 109.2 MB. *Time travel:* each of the six snapshots returns exactly the rows accumulated up to its year. *Schema evolution:* adding a column to a test table left its data file untouched. *Storage:* silver occupies 291.5 MB, against 2,803.0 MB for the CSV files of the six years, gold 139.5 MB and the three new warehouse tables 753.1 MB. *Reproducibility:* a rebuild from a clean clone, with a new Python environment, database and object store, finished in 569 s and returned the same SHA-256 for all seven queries; it reused source files already downloaded, after checking their hashes, because the publisher's server was too slow that night to fetch them again.

9.3 What the Records Show

Read through the warehouse, the records support a few descriptive findings and, more often, questions for the agencies that publish them; without ground truth, none of them establishes an irregularity. Georeferences are mostly close to their declared locality, at a median distance of 0.38 to 0.44 km depending on the year, but the share farther than 25 km fell from 2.6–3.7% in 2020–2022 to 0.2–0.4% from 2023 onwards. For 2022 the two official sources on municipal contracting disagree: 947 of 2,469 municipalities declared to the census fewer works contracts than SHCP lists with the municipality as contracting party, and 94 of them declared none. Temascaltepec is among the latter, with no contracts in the census against 16 worth 19.4 million pesos in SHCP. The universes differ —the census covers all municipal resources and SHCP only federal ones, and a contract listed in 2022 may have been signed earlier— so the comparison selects municipalities to ask, not to accuse.

The criteria behave on the 1,465,856 investment project-years as Section 7.4 anticipated. The conference C1 flags 2,010 of them and the per-type C1 57,797, computed here on unit cost —approved amount over physical target— within each classification and unit with at least 30 peers; only 491 are flagged by both, which confirms on real records that the original criterion selects large works rather than expensive ones. C2 flags 387,066, C3 48,943 of the 1,291,156 with both progress figures, and C4 74,002: review lists to rank for an oversight office, not detections. Spending follows need: in 2023 the median

amount paid per inhabitant on investment projects rises from 840 pesos in municipalities of very low deprivation to 4,268 pesos in those of very high deprivation, and the Spearman correlation between deprivation index and amount paid per inhabitant lies between .39 and .57 across years. Direct award accounts for 49.8% of the works contracts of 2023–2025, but for only 23.7% of their amount.

9.4 A Public Site Verified against the Databases

The results are published as a static map with a query page (Fig. 4), apart from the conference site so that the links printed in the chapter stay as they are. Each of its six layers is the stored result of one SQL file run on the warehouse or on the lake — once per state for the two project layers—, and a verification script repeats every query and compares the result byte by byte with the published file, failing if a file is edited by hand or if the databases stop returning the same bytes. The details of every municipality and project name the official file they come from, with its SHA-256, so that a reader can find the record in the source; 159,032 of the 253,346 projects of 2024 also list the links that the reporting agency published with their contracts, kept in an annex table of the lake and shown as reported. Projects load one state at a time and can be filtered by status or flag, a keyword search finds municipalities and projects, and each project can be compared in two releases of Esri's public satellite-imagery archive: the last one published before the project started and the latest. The base map tries a chain of key-free tile providers, each checked before use, lets the reader switch provider by hand and, when none answers, falls back to a national outline with state limits served by the site itself.

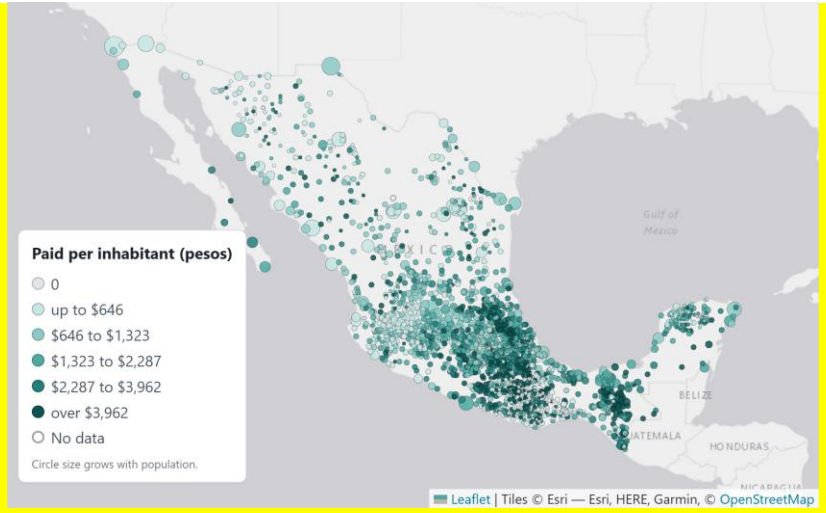


Fig. 4. The public map of the lakehouse layer: amount paid per inhabitant in 2024 by municipality, drawn at the municipal seat. Official records; every layer is the verified result of one query.

10 Conclusions and Future Work

10.1 Conclusions

This work presented a pragmatic architecture centered on a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complementing the analytical store with S3-compatible object storage for unstructured evidence under a single REST API. Returning to the research questions: RQ1 is answered affirmatively and, in this version, empirically: ten dimensions, five of them SCD 2, two fact tables and six views on commodity managed services, measured over the reference stack of Section 7.2, where the analytical layer answers in single-digit milliseconds and the limit lies in the delivery layer rather than in the dimensional design. The caveat is that the measurement is of a reference deployment under synthetic load, not of a production installation serving a municipality. RQ2 is answered in Section 4: an event-grain transaction fact table plus a work-month snapshot table, with SCD 2 restricted to the five audit-relevant entities, suffices to serve delay detection, budget alerting and participation analytics from one schema. RQ3 is answered in Section 7.4: the rule attains 0.46 precision at 0.35 recall against latent causes and is out-ranked by an unsupervised baseline, with its failures concentrated in cost anomalies, whose criterion is mis-specified.

The proposal shows that it is possible to build a transparency system with analytical capability (OLAP views, SCD 2 history), georeferenced photographic evidence, and citizen participation, without relying on high-cost enterprise platforms.

RQ4 is answered in Sections 8 and 9. The warehouse evolved into a lakehouse over 1,537,019 official project-year records without altering any of its original tables, and the two architectures proved complementary rather than rival: both returned identical results to every paired query, the lake was faster on national aggregations and the warehouse on queries restricted to one state or municipality. The real records also turned the Type 2 dimensions into a measured history, and showed how much

of what an infrastructure disclosure standard asks for the source does not record. The per-type criterion repairs part of the failure found in Section 7.4, but overpricing remains hard to detect from amounts alone.

10.2 Limitations

(1) There is no hosted dynamic deployment: permanence rests on the static artefact, and the API is distributed to be run by whoever needs it rather than kept online by us. (2) The participation module uses a simple voting scheme that may not adequately represent minorities. (3) The quality of the analysis depends on the quality, coverage, and freshness of the data entered by field supervisors. (4) All analytical results come from synthetic data —the warehouse population from a seeded Faker generator, the detection results from a process simulation over seeds 42 to 51— because privacy constraints preclude publishing municipal records; nothing here has been validated against real ones. Sections 8 and 9 use official records, but without ground truth: they test the architecture, not the detector. (5) The architecture is, in its current state, *data-warehouse-centric*: the object store holds binary evidence but is not queried analytically. Section 8 lifts this limitation for official records, though not yet for the binary evidence. (6) The public map endpoints read the operational schema rather than the warehouse views, so the two read paths can diverge; unifying them is pending. (7) Access control is lightweight —the staff role travels in a request header and the citizen session token, though HMAC-signed, has no expiry— which is adequate for a prototype whose analytical surface is public by design, but must be hardened before handling non-public data. (8) The official records cover only the federally funded spending reported to SHCP, not every work of a municipality; and the 2025 file, although SHCP's definitive report, holds 91,062 projects against 253,494 in 2024, for reasons not yet verified. (9) The lake was measured on one workstation, with PostgreSQL in its default configuration, and against SeaweedFS rather than R2. (10) The rebuild from a clean clone reused source files already downloaded.

10.3 Future Work

(0) Evolve the object layer toward a full lakehouse by adopting an open table format (Armbrust et al., 2020) (Apache Iceberg, or Parquet with transactional metadata) queryable directly over R2 through a lightweight engine such as DuckDB or Trino, without introducing Apache Spark; (1) a computer-vision inference pipeline to automatically classify work progress from the stored photographs; (2) publishing the schema as an OWL vocabulary to interoperate with Linked Data, and exposing an OC4IDS-conformant (Open Contracting Partnership & CoST – The Infrastructure Transparency Initiative, n.d.) projection of the warehouse so that the municipality's data becomes comparable with other CoST-aligned jurisdictions; (3) quadratic voting to improve the representativeness of minority proposals; (4) time-series models to predict completion dates; (5) recomputing the cost-anomaly criterion per work type, following the failure analysis of Section 7.4, and re-evaluating against the same ground truth; and (6) validating the architecture in additional municipalities, with a pre/post measurement of report-preparation time and information-request volume. This version implements item (0), on an S3-compatible store other than R2, and items (2) and (5) (Sections 8 and 7.5), and covers every municipality in the country without the pre/post measurement of item (6). Two items follow from it: (7) registering the w3id.org namespace and publishing the OC4IDS projection for every state, and (8) taking to SHCP and INEGI the records that Section 9.3 selects.

10.4 Availability

The warehouse DDL, SCD 2 triggers, seeded generators, load-testing scripts and the evaluation script are archived at https://github.com/gabrielhuav/PublicMunicipalWorks_DWH. A static demonstration is published through GitHub Pages at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/. It preserves the interface of the original prototype —four role workspaces, the citizen-participation module and the map— replacing remote authentication and writes with synthetic in-browser behaviour, and adds Spanish/English switching and two colour themes in light and dark mode; it makes no request to any backend service. Access is open by construction: the credentials are printed on the landing page and the form accepts any value, and every screen is served from a synthetic dataset held in the visitor's own tab, so no input can affect another visitor. The Flask implementation in `backend/` is the operational reference, with the same `DEMO-` accounts constrained to read-only. It is runnable rather than archived: `docker/` raises PostgreSQL and the API with the seeded warehouse, so an agency wanting to operate the system starts from a working stack, and the figures above were verified against that container. Table 2 is reproduced by running `eval_deteccion_v2.py` from `scripts/evaluacion/`, which drives the generator over seeds 42–51 and the four prevalences; the latencies of Section 7.2 by `scripts/medir_rendimiento.py` against the container, and the declared dimension types by `scripts/verificar_scd.py` against the schema. The population figures are constants in `scripts/generate_synthetic_data.py`, met by construction; its `--verificar` flag checks them without a database.

The lakehouse layer, its queries, both projections, the per-type criterion and the output of every measurement in Sections 7.5, 7.7, 8 and 9 are archived at https://github.com/gabrielhuav/PublicMunicipalWorks_Lakehouse, and its map and query page are published at https://gabrielhuav.github.io/PublicMunicipalWorks_Lakehouse/. One command, `lakehouse/etl/ejecutar.py`, downloads the nine source files, stops if one no longer matches its hash, builds the layers, loads the warehouse and compares the seven paired queries; `lakehouse/verificacion/verificar_capas.py` then checks the published site against both databases. The

per-type criterion is reproduced by `scripts/evaluacion/eval_criterio_costo.py` and the corrections of Section 7.7 by `scripts/cifras_capitulo.py`.

References

Ambituuni, A. (2026). Infrastructure transparency initiative and anticorruption in public infrastructure projects: A principal–agent perspective to the antecedents that enable (or hinder) the effectiveness of transparency. *Engineering, Construction and Architectural Management, 33*(7), 5934–5957. https://doi.org/10.1108/ECAM-02-2025-0273

Apache Software Foundation. (2025). *PyIceberg* (Version 0.9.1) [Computer software]. https://py.iceberg.apache.org/

Arenas-Guerrero, J. (2026). *morph-kgc* (Version 2.10.0) [Computer software]. https://github.com/morph-kgc/morph-kgc

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafrański, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., . . . Zaharia, M. (2020). Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment, 13*(12), 3411–3424. https://doi.org/10.14778/3415478.3415560

Barreto, A., Huang, Y. S., & Therianos, J. (2021, December 17). *Building a geospatial lakehouse, part 1*. Databricks Blog. https://www.databricks.com/blog/2021/12/17/building-a-geospatial-lakehouse-part-1.html

Consejo Nacional de Evaluación de la Política de Desarrollo Social. (n.d.). *Índice de rezago social por municipio, 2020* [Social deprivation index by municipality, 2020] [Data set]. datos.gob.mx. Retrieved November 2, 2026, from https://www.datos.gob.mx/dataset/ef697817-92a9-4937-8323-c08dda8bd00f/resource/862d0ad7-3c26-46dd-8e0d-7f2c49c88d1c/download/irs_municipal_2020.csv

DuckDB Foundation. (2025). *DuckDB* (Version 1.3.2) [Computer software]. https://duckdb.org/

Espinoza-Arias, P., Fernández-Ruiz, M. J., Morlán-Plo, V., Notivol-Bezares, R., & Corcho, O. (2020). The Zaragoza's knowledge graph: Open data to harness the city knowledge. *Information, 11*(3), Article 129. https://doi.org/10.3390/info11030129

Faisal, S., & Sarwar, M. (2014). Handling slowly changing dimensions in data warehouses. *Journal of Systems and Software, 94*, 151–160. https://doi.org/10.1016/j.jss.2014.03.072

Fazekas, M., & Kocsis, G. (2020). Uncovering high-level corruption: Cross-national objective corruption risk indicators using public procurement data. *British Journal of Political Science, 50*(1), 155–164. https://doi.org/10.1017/S0007123417000461

Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys, 54*(4), Article 71. https://doi.org/10.1145/3447772

Instituto Nacional de Estadística y Geografía. (n.d.-a). *Censo de Población y Vivienda 2020: Principales resultados por localidad (ITER)* [2020 Population and Housing Census: Main results by locality] [Data set]. Retrieved November 2, 2026, from https://www.inegi.org.mx/contenidos/programas/ccpv/2020/datosabiertos/iter/iter_00_cpv2020_csv.zip

Instituto Nacional de Estadística y Geografía. (n.d.-b). *Censo Nacional de Gobiernos Municipales y Demarcaciones Territoriales 2023: Contrataciones públicas* [2023 National Census of Municipal Governments: Public procurement] [Data set]. Retrieved November 2, 2026, from https://www.inegi.org.mx/contenidos/programas/cngmd/2023/datosabiertos/m2/contrata_publica_cngmd2023_csv.zip

Janowicz, K., Hitzler, P., Li, W., Rehberger, D., Schildhauer, M., Zhu, R., Shimizu, C., Fisher, C. K., Cai, L., Mai, G., Zalewski, J., Zhou, L., Stephen, S., Gonzalez, S., Mecum, B., Lopez-Carr, A., Schroeder, A., Smith, D., Wright, D., . . . Currier, K. (2022). Know, know where, KnowWhereGraph: A densely connected, cross-domain knowledge graph and geo-enrichment service stack for applications in environmental intelligence. *AI Magazine, 43*(1), 30–39. https://doi.org/10.1002/aaai.12043

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008)* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Lourenço, R. P. (2015). An analysis of open government portals: A perspective of transparency for accountability. *Government Information Quarterly, 32*(3), 323–332. https://doi.org/10.1016/j.giq.2015.05.006

Lu, C. (2025). *SeaweedFS* (Version 3.87) [Computer software]. https://github.com/seaweedfs/seaweedfs

Open Contracting Partnership, & CoST – The Infrastructure Transparency Initiative. (n.d.). *Open Contracting for Infrastructure Data Standard (OC4IDS)*. Retrieved August 1, 2026, from https://standard.open-contracting.org/infrastructure/latest/en/

Pellissier Tanon, T. (2023). *Oxigraph* (Version 0.3.22) [Computer software]. https://github.com/oxigraph/oxigraph

Rajabi, E., Midha, R., & de Souza, J. F. (2023). Constructing a knowledge graph for open government data: The case of Nova Scotia disease datasets. *Journal of Biomedical Semantics, 14*, Article 4. https://doi.org/10.1186/s13326-023-00284-w

Rivest, S., Bédard, Y., Proulx, M.-J., Nadeau, M., Hubert, F., & Pastor, J. (2005). SOLAP technology: Merging business intelligence with geospatial technology for interactive spatio-temporal exploration and analysis of data. *ISPRS Journal of Photogrammetry and Remote Sensing, 60*(1), 17–33. https://doi.org/10.1016/j.isprsjprs.2005.10.002

Secretaría de Hacienda y Crédito Público. (n.d.). *Gasto federalizado: Destino del gasto, 2020–2025* [Federally funded spending: Use of funds, 2020–2025] [Data set]. Transparencia Presupuestaria. Retrieved November 2, 2026, from https://www.transparenciapresupuestaria.gob.mx/

Villa Vargas, J. M., Hurtado Avilés, G., & Climent Hernández, J. A. (2026). Cuando México tiembla: La historia contada por los datos [When Mexico shakes: The story told by the data]. *AZCATL, 4*(6), 28–33. https://doi.org/10.24275/AZC2026E1004

Zaharia, M., Ghodsi, A., Xin, R., & Armbrust, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR 2021)*.

Declaration on the Use of Generative AI: The authors used a generative AI assistant, under their direction, for language editing, LaTeX formatting, drafting text subsequently revised and approved by the authors, and writing the auxiliary scripts distributed with the artefact, and generating the artefact's illustrative work images, each labelled as such in the file. The research questions, the dimensional design, the ETL, the data collection, the experiments and the interpretation of the results are the authors' own work. For this extended version they used Claude (Anthropic), under the same direction, to write and run the code of the lakehouse layer, the projections and the verification scripts, to trace each new figure to the script that produces it, and to draft the new sections, which the authors revised and approved.

Conflicts of Interest: The authors declare that they have no conflicts of interest relevant to the content of this article.